

Advanced NLP

Session 04 · Weights & Generation

Prof. Dr. Richard Sieg · TH Köln IWS · WS 26 / 27
Tuesday, October 13, 2026

From a stack of blocks to a model you can talk to

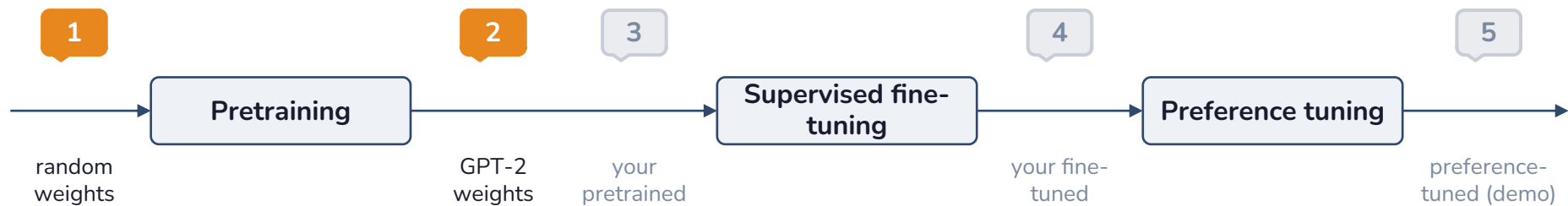
Time

15:30	Assembling GPT
15:45	Decoding: from scores to a word
15:58	Lab: notebook 04
16:28	Loading GPT-2's weights, the chat window
16:38	Lab: talk to GPT-2
16:50	Day 1, done

						Now ↓
Tue 13	01 Kick-off	02 Attention I	Lunch	03 Transformer	04 Generation	
Wed 14	05 Pretraining	06 Story of LLMs	Lunch	07 Post-training	08 Evaluation	
Thu 15	Hackathon		Lunch	Presentations		
	9:30			17:00		

Today you meet the first two of five models

- Random weights: noise.
- OpenAI's GPT-2 weights, in your own code: fluent text that ignores you.
- Tomorrow: your own pretrained model, your own fine-tuned model, and a preference-tuned model in a demo.



The model gives probabilities.
What it says is your choice.

Agenda

01.

Assembling GPT

02.

Decoding

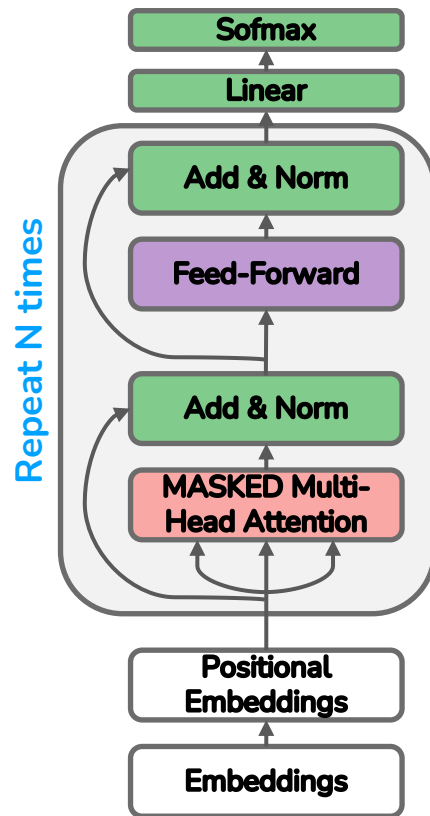
01.

Assembling GPT

The decoder from this afternoon is the whole model

- This afternoon you built one block. GPT is a stack of them, with the token table below and a linear layer on top.
- Our model stacks six. GPT-2 small stacks twelve, GPT-2's largest version forty-eight.

**We have built a
Transformer Decoder**



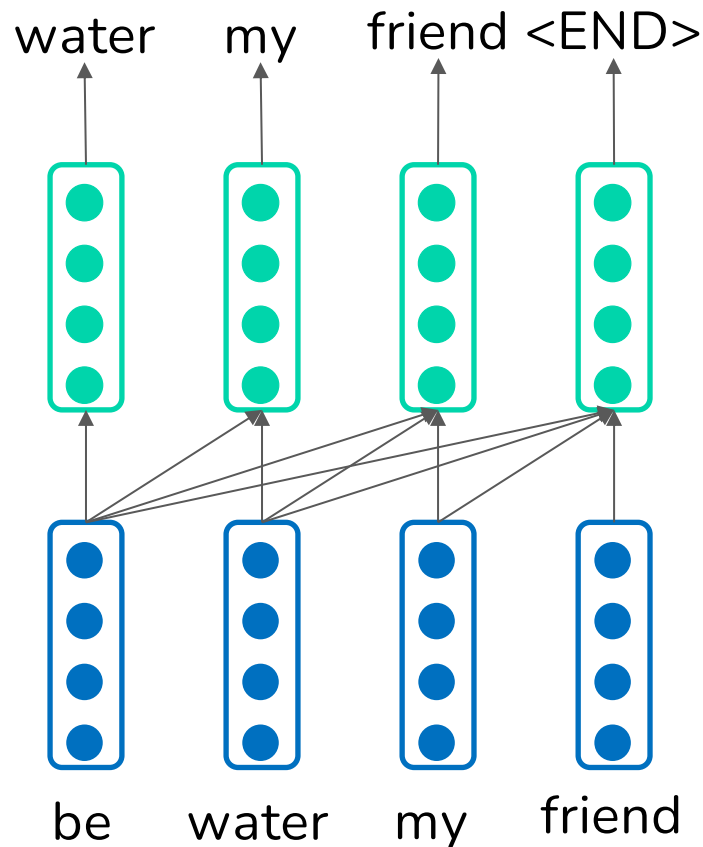
GPTModel turns 512 token IDs into $512 \times 50,257$ scores

Step	Shape
Token IDs	(b, 512)
Token table, tok_emb	(b, 512, 256)
plus the positional table, pos_emb, positions 0 to 511	(b, 512, 256)
Dropout	(b, 512, 256)
Six TransformerBlocks	(b, 512, 256)
Final LayerNorm	(b, 512, 256)
Output layer, out_head	(b, 512, 50,257)

- The positional table is the same for every sequence in the batch and broadcasts over it.
- The blocks sit in an `nn.ModuleList` and forward loops over them, so it can pass `attn_mask` along on Wednesday.

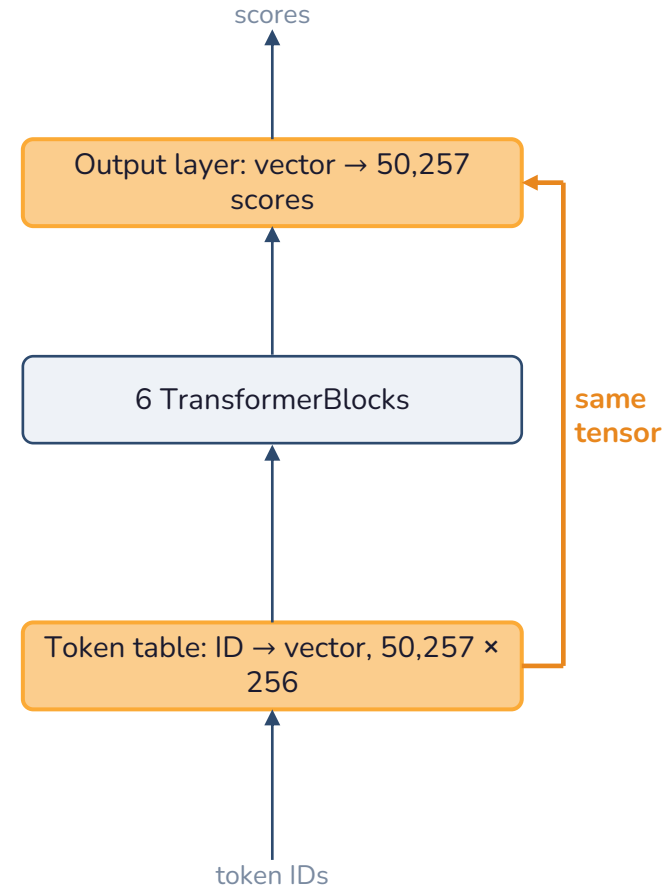
Every position predicts its own next token

- The output layer turns each position's vector into 50,257 scores, one for every token in the vocabulary.
- Training uses all positions at once, tomorrow morning. Generating only needs the last one.



The output layer is the token table, used a second time

- At the bottom, the token table turns an ID into a vector: 50,257 rows of 256.
- At the top, the same table turns a vector back into 50,257 scores.
- One tensor, not two. That saves 12.9 million parameters: 17.7 million instead of 30.6.
- GPT-2 ties them like this. Raschka's book keeps them separate, and says that trained better in his experience.



An untrained model should be unsure, not wrong

Initial loss

A uniform guess over 50,257 tokens	$\ln 50,257 = 10.8$
------------------------------------	---------------------

GPT-2's initialization, every weight from $N(0,0.02)$	about 10.9
---	------------

PyTorch's default for embeddings, $N(0,1)$	about 59
--	----------

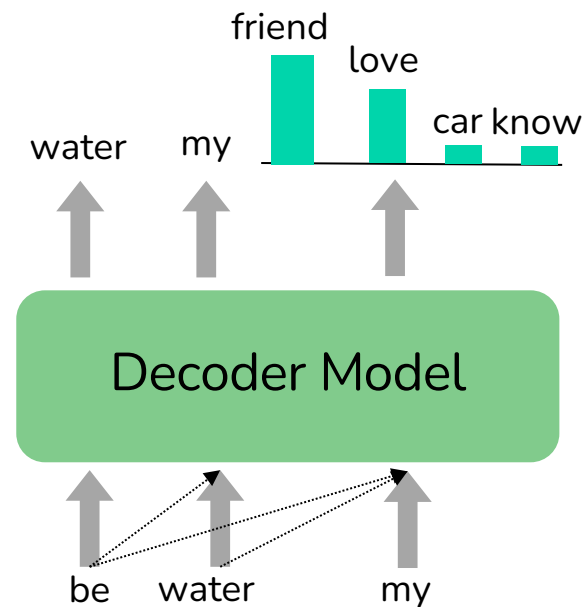
- An untrained model knows nothing, so it should start at the loss of a uniform guess.
- PyTorch draws embeddings with a standard deviation of 1. Through the tied output layer, that gives scores with a spread of 16: confident, random, wrong.
- GPT-2 uses 0.02 everywhere. That is what `_init_weights` does.

02.

Decoding

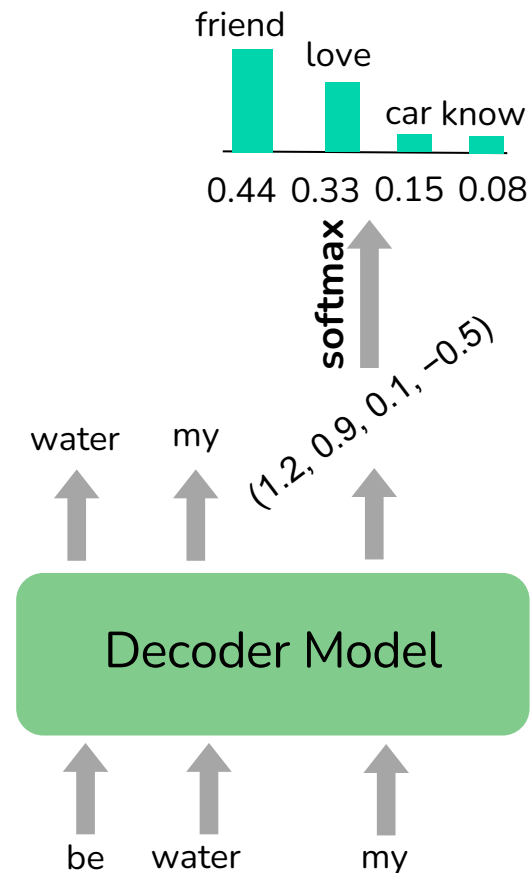
A decoder loops: predict, append, predict again

- A decoder is a transformer with masked (causal) attention: each position attends only to the tokens before it, so the model predicts the next token from the ones so far.
- Output: a probability distribution over the next token
- Trained on next-token prediction
- **Autoregressive**, we generate by looping:
 1. Sample a token from the distribution
 2. Append it to the context
 3. Predict again on the longer context
 4. Stop at the end-of-text token



Greedy decoding always picks the top token

- Decoding: picking a token from the predicted distribution.
- The network outputs logits: one raw score per vocabulary token
- Softmax turns logits into probabilities that sum to 1
- Decoding strategies differ in how they pick from that distribution
- **Greedy:** always pick the single most probable token
 - Simple, fast, but also deterministic and repetitive
- **Random sampling:** pick a token according to its probability
 - Not deterministic, more creative
 - Issue: long tail of odd, low probability words that could derail the text

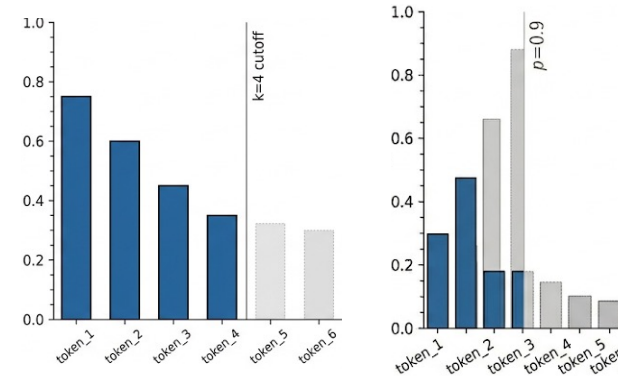
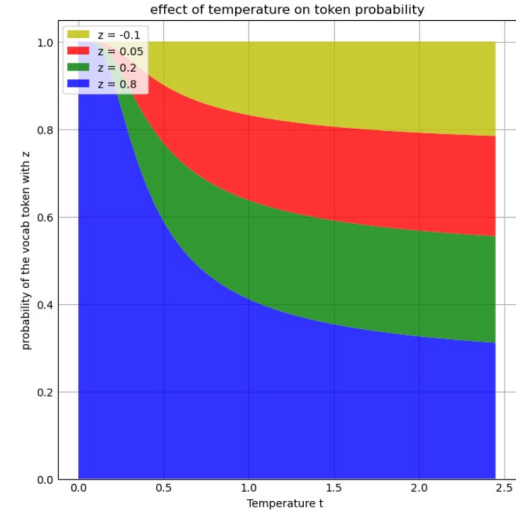


Temperature reshapes, top-k and top-p cut the tail

- **Temperature (T):** divide the logits by T , then apply softmax:

$$P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $T = 1$ unchanged, $T < 1$ sharper (toward greedy), $T \rightarrow 0$ greedy, $T > 1$ flatter and more random. Usually 0.7
- **Top-k:** keep the k highest-probability tokens, set the rest to zero, renormalize, then sample (fixed count)
- **Top-p (nucleus):** keep the smallest set whose probabilities add up to at least p , renormalize, then sample (fixed probability mass). Usually 0.9



One distribution, four decisions

“be water my ___”

	friend	love	car	know
Greedy	chosen			
Temperature 0.1	0.95	0.05	0.00	0.00
Temperature 1	0.44	0.33	0.15	0.08
Temperature 2	0.35	0.30	0.20	0.15
Top-k, k = 2	0.57	0.43	out	out
Top-p, p = 0.9	0.48	0.36	0.16	out

- Top-k always keeps k tokens. Top-p keeps as many as it takes to reach p: three here, more for a flat distribution, fewer for a peaked one.

The model said 0.44, 0.33, 0.15, 0.08. Which word gets written is decided outside the model.

Tutorial

Notebook 04: assemble GPT and let it generate

1. Implement GPTModel: `__init__`, `_init_weights` and `forward`.
 2. Check the parameter count: 17,731,328.
 3. Implement `generate`: greedy for temperature 0, otherwise temperature, top-k, top-p, then sample.
 4. Generate from your untrained model. Expect noise: checkpoint 1.
 5. The GPT-2 download cell does not depend on your code. If it has not run yet, run it now: it takes a while.
 6. Commit and push.
- Your model has 17,731,328 parameters
 - Temperature 0 gives the same output twice
 - Your untrained model generates noise

About 45 minutes, in pairs

Talk to GPT-2, and break it on purpose

1. Load GPT-2's weights into your `GPTModel`. The output should be English.
2. Open the chat window. Ask it a question, then start a story.
3. Set the temperature to 0. What happens after a few sentences?
4. Set it to 2. When does it stop making sense?
5. Back to 0.8, and top-p to 0.5. What changes?

About 12 minutes

- GPT-2 speaks English in your own code
- You have seen greedy decoding loop
- You have seen a high temperature drift into nonsense

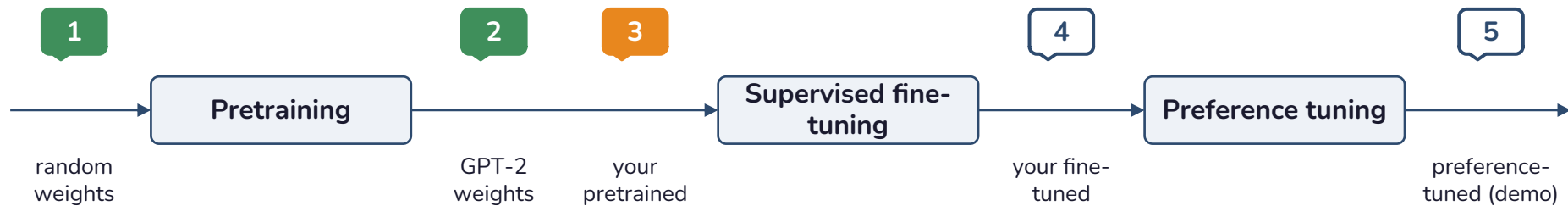
Today you went from text to a model that speaks

Block	You built
1	Tokens and training windows: <code>get_batch</code>
2	Attention
3	MultiHeadAttention, LayerNorm, FeedForward, TransformerBlock
4	GPTModel, generate, and GPT-2's weights in your own code

Everything except the weights is yours. Tomorrow the weights are too.

Tomorrow, checkpoint 3: your own weights

- Pretraining on TinyStories on your own GPU: about 150 million tokens in twelve minutes.
- Then how GPT-2 turned into ChatGPT, told as changes to the model you built.
- In the afternoon, your model learns to follow instructions.



Sources

Literature

- Raschka, Build a Large Language Model (From Scratch), section 4.6: GPTModel and weight tying, 163 million untied and 124 million tied. Section 4.7: generating text. Section 5.3: greedy decoding, temperature, top-k. Section 5.5: loading OpenAI's weights.
- Holtzman et al. 2019, The Curious Case of Neural Text Degeneration, arxiv.org/abs/1904.09751. Top-p, and greedy decoding's loops.
- Radford et al. 2019, Language Models are Unsupervised Multitask Learners. GPT-2's sizes.